GRADUATION PROJECT PRESENTATION

Online Medicine Ordering System

An architectural overview of a modern web portal linking patients with active pharmacies using live database updates.

Laravel 10 | MySQL | Bootstrap 5 | PHP 8.2 MVC

System Concept & Core Objectives

The Challenge vs. Solution

The Problem: Patients face severe difficulties requesting vital prescriptions without physical travel, alongside a critical lack of real-time stock feedback.

The Solution: A robust, real-time MVC platform running on relational databases, enabling instant checking, quick automated checkout, and responsive stock deductions.

Key Objectives Built

- **Frictionless Ordering:** Fast order pipeline with guest session tracking.
- **Live Stocks:** Every order automatically decreases inventory instantly.
- **Admin Terminal:** Centralized controls for inventory and order cycles.
- **Aesthetic UX:** Highly intuitive user flows with localized styles.

Architectural Layout & Tech Stack

Model-View-Controller (MVC) Implementation

The system separates business rules from database structures and frontend interfaces using Laravel's core architecture pattern, guaranteeing scalability and seamless code maintainability.

- **Backend Framework: Laravel 10**
- **Database: MySQL locally on XAMPP**
- **Architecture: MVC & Eloquent ORM**
- **Frontend: Blade & Bootstrap 5**



Relational Database Blueprint

Entity Model	Primary Schema Properties	Relational Rules & Constraints
Customers	id, name, phone, address, timestamps	Has Many Orders (1 : N Relationship)
Medicines	id, name, quantity, price, timestamps	Has Many Orders (1 : N Relationship)
Orders	id, customer_id (FK), medicine_id (FK), quantity_requested, status	Belongs to One Customer / One Medicine (Resolves N : 1)
Admins	id, name, email, password, timestamps	Isolated access terminal for pharmacy staff

* Note: Schema generation is fully automated via Laravel Migrations using absolute constraint validation rules (e.g. onDeleteCascade cascade mappings).

Application Controller Logic & Session Flow

100%

SEAMLESS GUEST SESSIONS

$$Q_{requested} \leq Q_{available}$$

Validation Requirement Formula

Patient Routing Flow

Guest Tracking: To prevent forcing patients to set up tedious passwords, users are tracked during active checkouts via highly secure browser session cookies mapping back to `customer_id` database records.

Strict Double Validation: When an order hits `OrderController::store()`, the application executes successive inventory queries: first validating if the medication exists in lookup tables, and secondly assuring that total quantity exceeds requested values before executing deductions.

Dashboard Administration & Security Hardening

Custom Admin Guard

Isolated authentication structures and Custom Middleware blocks intercept all incoming dashboard traffic, safely redirecting guests back to admin login gateways.

Stock Control Terminal

An administrative CRUD dashboard panel facilitates rapid editing of medical lists, directly syncing modifications into the buyer search views without delay.

Security Measures

Guaranteed application defense through active CSRF tokens, strict controller-level model validation rules, and secure Bcrypt password hashing algorithms.

Engineering Challenges & Resolutions

⚠ Challenging Namespace Issues

The Challenge: Text editors occasionally saved critical PHP classes with unformatted Byte Order Mark signatures (UTF-8 with BOM), producing structural syntax errors on class namespaces.

The Solution: Constructed a local PowerShell build utility script to verify and clean script assets, programmatically formatting code files as standardized UTF-8 files prior to execution.

🛠 Database Cache Conflicts

The Challenge: Relational queries periodically default back to obsolete database settings during local tests due to persistent configuration cache storage.

The Solution: Systematically executing optimization sweeps during development loops using custom helper configurations:

```
php artisan config:clear
```

Project Roadmap & Core Conclusions

1. Order Statuses

Integrate live order pipelines: In-Process, Shipped, and Delivered.

2. Pay Gateway

Embed stripe digital checkout platforms for automated web transactions.

3. Live Alerts

Launch immediate SMS/Email dispatch triggers for order status updates.

4. Complete APIs

Provide secure API JSON endpoints to connect mobile platforms seamlessly.

Conclusion: This graduation project successfully establishes a responsive, MVC-driven solution resolving medication access challenges with optimized database structures, secure middleware, and live data integrity.

Image Sources



https://www.bdtask.com/sp/pharmacare/img/pharmacy-management-system.webp

Source: www.bdtask.com